

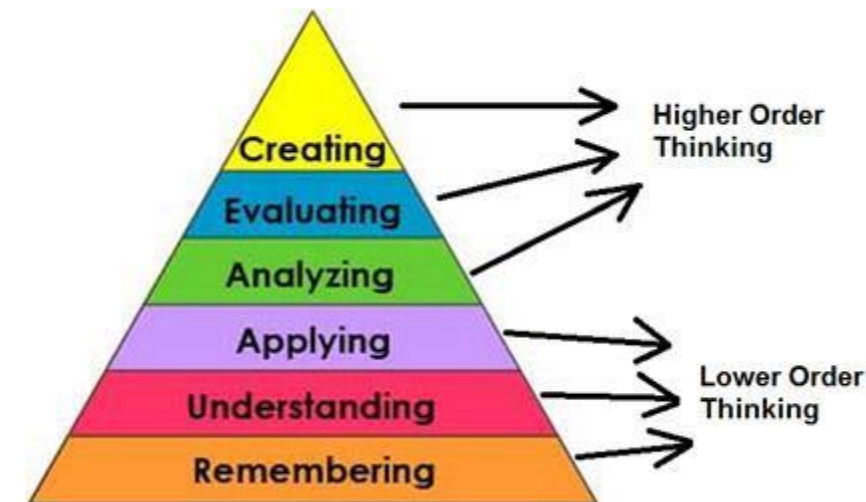
Kỹ Thuật Lập Trình Python

CHƯƠNG 7 XỬ LÝ NGOẠI LỆ



Mục tiêu

- Áp dụng kỹ thuật xử lý ngoại lệ để giải quyết lỗi phát sinh



- Ngoại lệ là gì? (Exception?)
- Tầm quan trọng khi xử lý ngoại lệ
- Cách xử lý ngoại lệ
- Lệnh assert
- Lệnh try

exception là gì?

- Phát triển hệ thống lớn => không thể nào kiểm soát hết tất cả các tình huống phát sinh, các kịch bản lỗi đáng tiếc có thể xảy ra.
- Các vấn đề về lỗi phát sinh trong chương trình gọi là ngoại lệ hoặc điều kiện bất thường.
- Nếu các ngoại lệ/điều kiện bất thường đó liên tục xảy ra thì hệ thống/chương trình sẽ ngừng thực thi (code bị lỗi, Python ngừng thông dịch).
- Xử lý ngoại lệ là phương pháp vô hiệu hóa các điều kiện bất thường có thể xảy ra.

Tầm quan trọng của exception?

- Chương trình được xử lý ngoại lệ tốt thì chúng giảm thiểu rủi ro.
- Dừng đột ngột chương trình.
- Chống thất thoát dữ liệu khi xử lý...

exception

Ví dụ

Sau đó, bình luận
kết quả nhận được
khi thông dịch.

```
def tinh(x, y):  
    kq = x / (x - y)  
    return kq
```

```
tinh(7, 9)
```

```
-3.5
```

```
tinh(8, 8)
```

```
-----  
ZeroDivisionError                                Traceback (most recent call last)  
<ipython-input-4-cd14c2078333> in <module>  
----> 1 tinh(8, 8)  
  
<ipython-input-1-418e8f29e47f> in tinh(x, y)  
      1 def tinh(x, y):  
----> 2     kq = x / (x - y)  
      3     return kq  
  
ZeroDivisionError: division by zero
```

Các loại ngoại lệ:

- <https://docs.python.org/3/library/exceptions.html>

Lệnh assert trong xử lý ngoại lệ

- Khi gặp lệnh `assert`, Python kiểm tra biểu thức kèm theo, nếu kết quả trả về là `False` thì Python tạo ra một exception có tên là `AssertionError`.
- Cú pháp:

`assert` `Expression` [, `Arguments`]

- `assert` là một lệnh dùng để kiểm nghiệm kết quả. Kết quả kiểm tra biểu thức trả về gồm hai giá trị `True` hoặc `False`.
- Nếu là `False` thì sẽ tạo ra một exception.
- Và thông thường lệnh này được dùng kiểm tra khả năng phù hợp của các input.

Minh họa assert:

- Gọi x, y là hai khoảng cách địa lý. Nếu user nhập vào khoảng cách có giá trị âm thì chương trình sẽ bắt lỗi.

```
x = float(input("Nhập khoảng cách X = "))
```

Nhập khoảng cách X = 90.8

```
y = float(input("Nhập khoảng cách Y = "))
```

Nhập khoảng cách Y = -18.5

```
assert x >= 0 and y >= 0, 'Khoảng cách vô lý'
```

```
-----  
AssertionError                                Traceback (most recent call last)  
<ipython-input-14-84fd1661088b> in <module>  
----> 1 assert x >= 0 and y >= 0, 'Khoảng cách vô lý'
```

AssertionError: Khoảng cách vô lý

Kỹ thuật xử lý exception

- **try ... except ... else ...** (nếu KO lỗi thì thực hiện lệnh **else**)
- **try ... except ... finally ...** (lệnh trong **finally** luôn thực thi)
- **try ... finally ...** (Khối lệnh này sẽ thực hiện trước)
- Tự tạo riêng exception (Custom Exception)

Sử dụng `try ... except ... else ...`

Dạng #1

try:

Thực hiện các lệnh trong chương trình tại đây.
Các lệnh tại đây có khả năng xảy ra lỗi.

except Exception_1:

Nếu có lỗi xảy ra thì bắt lệnh này.

except Exception_2:

Nếu có lỗi xảy ra thì bắt lệnh này.

else:

Nếu không có lỗi xảy ra thì thực hiện lệnh này.

Sử dụng `try ... except ... else ...`

Dạng #2

`try:`

Thực hiện các lệnh trong chương trình tại đây.
Các lệnh tại đây có khả năng xảy ra lỗi.

`except:`

Nếu có lỗi xảy ra thì bắt lệnh này.

`except:`

Nếu có lỗi xảy ra thì bắt lệnh này.

`else:`

Nếu không có lỗi xảy ra thì thực hiện lệnh này.

Sử dụng **try ... except ... else ...**

Dạng #3

try:

Thực hiện các lệnh trong chương trình tại đây.

Các lệnh tại đây có khả năng xảy ra lỗi.

except (Exception_1, Exception_2... Exception_n) :

Nếu có lỗi xảy ra thì bắt lệnh này.

else:

Nếu không có lỗi xảy ra thì thực hiện lệnh này.

```
try:  
    tinh(8, 8)  
except ZeroDivisionError as zde:  
    print('Lỗi: ', zde)
```

Lỗi: division by zero

Sử dụng try ... except ... else ...

VD minh họa (không có exception)

```
import math
while(True):
    n = float(input("Nhập vào một số bất kỳ: "))
    sqp_n = math.sqrt(n)
    print(f"Căn bậc hai của {n} bằng {sqp_n}")
```

Kết quả (khi không sử dụng exception):

```
Nhập vào một số bất kỳ: 9
Căn bậc hai của 9.0 bằng 3.0
Nhập vào một số bất kỳ: -9
Traceback (most recent call last):
  File "C:/Projects/PyPB/main.py", line 4, in <module>
    sqp_n = math.sqrt(n)
ValueError: math domain error
```

Sử dụng try ... except ... else ...

VD minh họa (có sử dụng exception)

```
import math
while(True):
    n = float(input("Nhập vào một số bất kỳ: "))
    try:
        sqp_n = math.sqrt(n)
        print(f"Căn bậc hai của {n} bằng {sqp_n}\n")
    except ValueError as v1:
        print("Chương trình bị lỗi:", v1)
        print("Phót lò và tiếp tục...\n")
```

Kết quả:

```
Nhập vào một số bất kỳ: 8
Căn bậc hai của 8.0 bằng 2.8284271247461903

Nhập vào một số bất kỳ: -8
Chương trình bị lỗi: math domain error
Phót lò và tiếp tục...

Nhập vào một số bất kỳ: 26
Căn bậc hai của 26.0 bằng 5.0990195135927845

Nhập vào một số bất kỳ:
```

Sử dụng try ... except ... finally ...

VD minh họa (có sử dụng exception)

```
while True:
    str_value = input("Nhập vào giá trị N = ")
    try:
        print(f"Đã nhập vào {str_value}. "
              f"Kiểu ", type(str_value))
        value = int(str_value)
        print("Chuyển string sang int:", value,
              f"Kiểu {type(value)}")
    except ValueError as ve:
        print("Đã xảy ra lỗi:", ve)
    finally:
        print("Giá trị N hiện tại là " + str_value)
        str_value = "Rỗng"
        print("Giá trị N được gán lại:", str_value)
    print("")
```

Sử dụng try ... except ... else ...

VD minh họa (có sử dụng exception)

```
Nhập vào giá trị N = 88
Đã nhập vào 88. Kiểu <class 'str'>
Chuyển string sang int: 88 Kiểu <class 'int'>
Giá trị N hiện tại là 88
Giá trị N được gán lại: Rỗng

Nhập vào giá trị N = tám
Đã nhập vào tám. Kiểu <class 'str'>
Đã xảy ra lỗi: invalid literal for int() with base 10:
'tám'
Giá trị N hiện tại là tám
Giá trị N được gán lại: Rỗng

Nhập vào giá trị N =
```

Sử dụng **try ... except ... finally ...**

Cú pháp chung

try:

Thực hiện các lệnh trong chương trình tại đây.
Các lệnh tại đây có khả năng xảy ra lỗi.

except Exception_1:

Nếu có lỗi xảy ra thì bắt lệnh này.

except Exception_2:

Nếu có lỗi xảy ra thì bắt lệnh này.

finally:

Luôn luôn thực hiện lệnh này (bất kể việc bắt được lỗi hay không bắt được lỗi).

Sử dụng try ... except ... finally ...

```
while True:
    str_value = input("Nhập vào giá trị N = ")
    try:
        print(f"Đã nhập vào {str_value}. "
              f"Kiểu ", type(str_value))
        value = int(str_value)
        print("Chuyển string sang int:", value,
              f"Kiểu {type(value)}")
    except ValueError as ve:
        print("Đã xảy ra lỗi:", ve)
    finally:
        print("Giá trị N hiện tại là " + str_value)
        str_value = "Rỗng"
        print("Giá trị N được gán lại:", str_value)
    print("")
```

Sử dụng `try ... except ... finally ...`

```
Nhập vào giá trị N = 88
Đã nhập vào 88. Kiểu <class 'str'>
Chuyển string sang int: 88 Kiểu <class 'int'>
Giá trị N hiện tại là 88
Giá trị N được gán lại: Rỗng

Nhập vào giá trị N = tám
Đã nhập vào tám. Kiểu <class 'str'>
Đã xảy ra lỗi: invalid literal for int() with base 10:
'tám'
Giá trị N hiện tại là tám
Giá trị N được gán lại: Rỗng

Nhập vào giá trị N =
```

Cú pháp chung:

`try:`

Thực hiện các lệnh trong chương trình tại đây.
Các lệnh tại đây có khả năng xảy ra lỗi.

`finally:`

Khối lệnh này sẽ thực hiện trước.

Sử dụng try ... finally ...

```
str_value = input("Nhập vào giá trị N = ")
try:
    print(f"Đã nhập vào {str_value}. "
          f"Kiểu ", type(str_value))
    value = int(str_value)
    print("Chuyển string sang int:", value,
          f"Kiểu {type(value)}")
finally:
    print("Giá trị N hiện tại là " + str_value)
    str_value = "Rỗng"
    print("Giá trị N được gán lại:", str_value)
```

Sử dụng try ... finally ...

```
Nhập vào giá trị N = sai
Traceback (most recent call last):
Đã nhập vào sai. Kiểu <class 'str'>
Giá trị N hiện tại là sai
  File "C:/PythonProjects/PyPB/test.py", line 5, in
<module>
Giá trị N được gán lại: Rỗng
    value = int(str_value)
ValueError: invalid literal for int() with base 10: 'sai'
```

Câu hỏi đặt ra:

- Có thể kết hợp **else** và **finally** trong quá trình bắt lỗi được không?

Tự tạo riêng exception (Custom Exception)

- Ngoài những ngoại lệ chuẩn có sẵn trong Python, chúng ta có thể tạo ngoại lệ cho riêng mình bằng cách kế thừa từ lớp Exception và sử dụng lệnh raise trong việc tạo exception.
- **SV tự tìm hiểu thêm.**

BÀI TẬP TỔNG HỢP

BÀI TẬP TỔNG HỢP NÂNG CAO

MINH HỌA VÉ SỐ VIETLOT 6/45

Người mua:

- Chọn 6 số bất kỳ từ 1 đến 45, các số không trùng nhau trong 1 vé. Gọi là 1 dãy số trên 1 vé. Ví dụ 10-18-26-33-39-44
- Có thể mua nhiều vé số tự chọn (n vé). Các vé có thể trùng dãy số với nhau.
- Giá bán mỗi vé 10.000 đ

Máy xổ số:

- Random 6 số bất kỳ từ 1 đến 45, các số không trùng nhau. Gọi là dãy số trúng thưởng.

Code it!

Dò kết quả: So sánh dãy số người mua và dãy số trúng thưởng:

- Nếu trùng nhau 3 con số thì người chơi nhận: 30.000 đ
- Nếu trùng nhau 4 con số thì người chơi nhận: 300.000 đ
- Nếu trùng nhau 5 con số thì người chơi nhận: 10.000.000 đ
- Nếu trùng nhau 6 con số thì người chơi nhận: 10.000.000.000 đ

Thống kê kết quả người chơi nhận được bao nhiêu tiền khi dò kết quả.

Người mua:

Mua số: 10-18-26-33-39-44

Máy xổ:

Dãy số: 10-19-22-39-40-44

Kết quả:

Bạn trúng: 30000



Thank you

Hỏi - Đáp

